DeadlineDesk

Week 7 Prototype Stage Report

A campus web system for placement rounds and academic assignment deadlines

Team member	Roll number	Primary responsibility
Aryan Gupta	1024030764	Architecture, CI/CD, Placement Track
Aksh Goyal	1024030766	Academic Dropbox, late-policy tests
Naveen Bansal	1024030767	UI flows, documentation, demo

Submitted to: Mr. Jeelani Asif
Date: 25 August 2026

Executive Summary

DeadlineDesk now delivers the two thin end-to-end paths promised in the Week 4 proposal. A Placement Admin can publish a company round with a required checklist and T-24h reminder log; a Student can complete the checklist and receive a readiness result. A TA can publish an assignment with a late policy; a Student can upload work and receive an automatic late outcome and similarity-stub score; the TA can review and grade the submission.

The implementation uses Django and SQLite, includes 12 automated tests, and is accompanied by SRS, design, testing, backlog, demo, CI, and weekly journal evidence.

Prototype Objectives and Status

ID	Objective	Status
O1	Role-based authentication for Student, TA / Faculty, and Placement Admin	Complete
O2	Company/round -> checklist -> T-24h reminder log	Complete
O3	Assignment policy -> submission -> late flag -> similarity stub -> grade	Complete
O4	At least five fixed black-box policy tests with 100% agreement	Complete: 6/6
O5	CI and published technical documentation	Complete

Scope Boundary

The Week 7 reminder is a persistent database log, not outbound email. Similarity is transparent token overlap for text, not an NLP plagiarism system. PostgreSQL and hosted deployment remain later increments.

Architecture and Technology

The prototype is a layered Django monolith. Server-rendered templates and forms call role-protected views; views orchestrate domain services and Django ORM models; SQLite and local media provide prototype persistence. The late-policy evaluator and similarity stub are isolated in the service layer so policy tests do not depend on page rendering.

Layer	Week 7 responsibility
Presentation	Responsive HTML/CSS, labeled forms, messages, empty/error states, light/dark tokens
Application	Session login, role decorator, validation, transactions, audit events
Domain	Users, placement entities, assignment/submission/grade, policy services
Persistence	Django ORM, SQLite, local uploads, versioned migrations
Quality	Pytest, pytest-django, system checks, GitHub Actions

Why Django

The prototype needs secure sessions, CSRF, role guards, forms, file uploads, migrations, and tests more than it needs separate frontend complexity. Django supplies mature components for those risks and keeps the implementation easy to demonstrate.

Core Data Constraints

ChecklistCompletion is unique per Student and ChecklistItem. Submission is unique per Student and Assignment. Grade is one-to-one with Submission. Database constraints back up view-level validation.

Implemented Workflows

Placement Track

The Placement Admin creates a Company and draft PlacementRound with open/close times, then adds ordered required or optional checklist items. Publishing requires at least one required item and creates a reminder scheduled for closing time minus 24 hours. Students cannot view drafts. Effective published/open/closed state is calculated from server time.

Admin -> company -> draft round -> required checklist -> publish -> T-24h reminder

A Student toggles checklist items. Readiness becomes true only when every required item is complete. Completions are isolated per student.

Academic Dropbox

The TA publishes an Assignment with due time and one policy: reject late work, accept with a configured penalty, or accept within a grace window. The Student uploads one file up to 5 MB. Server time is authoritative. Accepted work stores late outcome and penalty; rejected work creates no Submission.

TA -> assignment and policy -> Student upload -> late decision -> similarity stub -> review -> grade

For UTF-8 text, the similarity stub reports maximum Jaccard token overlap against earlier submissions to the same assignment. Binary or unsupported text returns 0. The interface explicitly states that the number is not a plagiarism verdict.

Policy Rules and Validation

Condition	Decision	Database effect
Submission time <= due time	On time; 0% penalty	Create Submission
Reject policy after due time	Rejected	No Submission
Penalty policy after due time	Late; configured penalty	Create Submission
Grace policy within boundary	Accepted in grace	Create Submission
Grace policy after boundary	Rejected	No Submission
Unknown policy	Fail closed with error	No Submission

Validation and Security Controls

Control	Implementation
Authentication	Django password hashing and server-side sessions
Authorization	Role decorator returns HTTP 403 for disallowed actions
Request integrity	CSRF middleware and tokens on state-changing forms
Data validation	Time ordering, policy fields, upload size/type, score maximum
Transactions	Round publish/reminder and grade/status update are atomic
Audit	Actor, action, entity, detail, and timestamp persisted

Test and CI Evidence

The fixed suite contains 12 tests. All pass locally with zero Django system-check issues.

Area	Evidence	Result
Late policy	Exact deadline, one-second late reject, penalty, grace boundary, after grace, unknown policy	6/6 pass
Authorization	Student request to admin creation route	HTTP 403
File privacy	Student request for another student's upload	HTTP 403
Placement	Publish creates exact T-24h reminder; checklist toggle	Pass
Academic	Late-penalty upload through TA grade; rejected attempt creates no row	Pass
Django	System check and migration drift check	Clean

Continuous Integration

On pushes and pull requests, GitHub Actions uses Python 3.12, installs pinned requirements, runs the Django system check, checks for migration drift, applies migrations, and runs pytest. A separate MkDocs workflow deploys the project documentation.

Browser Validation

All three seeded roles were tested through the rendered application. The Student reached 'Ready to apply' after three required documents. Academic lists, TA review surfaces, Placement Admin reminder logs, and effective round status were inspected. QA covered desktop and 390 x 844 mobile. No console warnings or errors remained.

Risks and Improvement Plan

Limitation	Next action
TA authorization is global	Add course/section ownership before pilot deployment
SQLite and local media	Deploy staging with PostgreSQL and managed storage
Reminder is a log only	Add a retry-safe background email worker
Similarity is a text stub	Keep the label; evaluate an approved service only if needed
One submission only	Define resubmission policy and immutable versions
No malware scanning	Scan content before accepting real student files

Team Contribution

Aryan Gupta: architecture, role core, Placement Track, CI/CD, integration.
Aksh Goyal: Academic Dropbox, policy rules, grading, black-box tests.
Naveen Bansal: role flows, responsive UI, documentation, browser QA, demo script.

Conclusion

DeadlineDesk meets the Week 7 objectives with two demonstrable workflows, explicit scope boundaries, repeatable demo data, automated policy evidence, and continuous integration. The implementation remains small enough to explain and test while providing a credible base for Week 8 security and deployment work.

Week 8 priority: authorization hardening and staging deployment before feature expansion.